UNIVERSITÀ DEGLI STUDI DI PERUGIA

Dipartimento di Matematica e Informatica

Progetto Introduzione Intelligenza Artificiale

Uniform Coloring

Autore: Hlade Carlos Floriano (367587)

A.A. 2025/2026

Indice

1 Descrizione formale del dominio e dei vincoli **3**
1.1 Descrizione formale del dominio 3
1.2 Descrizione formale dei vincoli 3

2 Struttura del programma **3**

3 Rete Neurale (Computer Vision) **4**
3.1 Preparazione del Dataset 4
3.2 Architettura del Modello 4

4 Image processing e costruzione degli stati **6**
4.1 Estrazione della griglia mediante Morfologia Matematica 6
4.2 Ritaglio intelligente e normalizzazione (Auto-Crop) 7

5 Algoritmi di ricerca e Solutore del Problema **7**
5.1 Modellazione del dominio (Problem) 7
5.2 Euristica (h_combined_cost) 8
5.3 Algoritmi utilizzati 9

6 Esecuzione e Statistiche dei Risultati **9**
6.1 Esecuzione del programma 9
6.2 Risultati Rete Neurale 11
6.3 Prestazioni degli Algoritmi di Ricerca 12
6.3.1 Matrice 2x2 13
6.3.2 Matrice 2x3 13
6.3.3 Matrice 2x4 14
6.3.4 Matrice 2x5 14
6.3.5 Matrice 3x4 15

1 Descrizione formale del dominio e dei vincoli

1.1 Descrizione formale del dominio

Uniform Coloring è un dominio di pianificazione e ricerca in cui si ha a disposizione una griglia rettangolare composta da celle da colorare e una "testina" colorante (indicata con 'T'). Ogni cella, ad eccezione di quella iniziale della testina, possiede un colore di partenza fra tre opzioni disponibili: Blu (B), Yellow (Y) e Green (G). L'agente (la testina) può muoversi all'interno della griglia nelle quattro direzioni cardinali (Nord, Sud, Est, Ovest) con un costo uniforme pari a 1 per ogni spostamento. La testina ha inoltre la capacità di cambiare il colore della cella in cui si trova, applicando uno dei tre colori disponibili.

L'obiettivo finale (goal) è raggiungere una configurazione in cui tutte le celle della griglia abbiano lo stesso colore (non importa quale esso sia) e la testina sia tornata esattamente nella sua posizione di partenza. I costi delle azioni di colorazione sono differenziati: $cost(B) = 1$, $cost(Y) = 2$, $cost(G) = 3$.

1.2 Descrizione formale dei vincoli

Le azioni eseguibili dall'agente e le regole del dominio devono rispettare i seguenti vincoli:

- L'agente si può muovere solo fra celle adiacenti (Nord, Sud, Est, Ovest) e non in diagonale.
- L'agente non può compiere movimenti che lo portino al di fuori dei confini fisici della griglia rettangolare.
- La testina colora la cella su cui si trova in quel momento. Durante lo spostamento, la cella abbandonata assume il colore appena applicato.
- La cella di partenza della testina (Start Position) non ha un colore proprio iniziale. Per definizione del dominio implementato, la testina non colora mai la propria casella di partenza.
- Il costo delle azioni si accumula ad ogni passo: gli spostamenti costano sempre 1, mentre i costi di colorazione dipendono dal pigmento scelto.
- L'agente può compiere solo un passo alla volta.

2 Struttura del programma

Il progetto è stato sviluppato in Python, separando logicamente la fase di classificazione visiva dalla ricerca della soluzione. I file principali sono:

- `requirements.txt`: file necessario per l'installazione delle librerie necessarie (TensorFlow, OpenCV, Numpy, AIMA, Matplotlib, ImageIO, ecc.).
- `keras_CNN.py`: script dedicato al caricamento del dataset EMNIST, al pre-processing dei dati, alla costruzione, addestramento e salvataggio della Rete Neurale Convoluzionale (CNN).
- `UniformColoring.py`: è il modulo principale che unisce la computer vision per l'estrazione della griglia, la definizione del problema (come sottoclasse `Problem` della libreria AIMA) e l'esecuzione degli algoritmi di ricerca.

3 Rete Neurale (Computer Vision)

Per riconoscere il contenuto delle celle (le lettere B, G, T, Y) è stata addestrata una Rete Neurale Convoluzionale (CNN) sul dataset EMNIST-letters.. Poiché il dominio richiede di riconoscere lettere scritte a mano all'interno delle celle della griglia, una CNN risulta l'architettura ideale grazie alla sua capacità di estrarre e combinare pattern visivi (feature) in modo gerarchico.

3.1 Preparazione del Dataset

Il dataset originale contiene l'intero alfabeto, ma per il nostro dominio ci interessano solo quattro classi specifiche. I dati sono stati perciò sottoposti a un rigoroso pre-processing:

- **Filtraggio e Rimappatura:** Sono state estratte solo le immagini corrispondenti agli indici di B (2), G (7), T (20) e Y (25). Le etichette sono state rimappate in un nuovo range $[0, 3]$ per adattarsi al classificatore finale.
- **Trasposizione e Reshape:** Le immagini di EMNIST si presentano ruotate e specchiate nel file `.mat`. È stata perciò applicata una trasposizione degli assi per riportare le lettere al corretto orientamento visivo, garantendo una forma finale di $28 \times 28 \times 1$ (immagini in scala di grigi a singolo canale).
- **Normalizzazione e Codifica:** I valori dei pixel sono stati normalizzati scalandoli dal range $[0, 255]$ all'intervallo continuo $[0, 1]$, facilitando la convergenza dei pesi durante la backpropagation. Le etichette sono state codificate tramite *One-Hot Encoding*.

3.2 Architettura del Modello

L'architettura costruita è di tipo sequenziale. È stata progettata su misura per estrarre feature robuste da caratteri manoscritti, gestendone le inevitabili irregolarità. Di seguito è riportata l'analisi dettagliata del flusso dei dati all'interno della rete:

- **Layer di Input e Data Augmentation:** La rete riceve in input tensori di dimensione $28 \times 28 \times 1$. Per rendere il modello invariante alle piccole imprecisioni della scrittura umana, i primi layer applicano trasformazioni casuali (attive solo durante la fase di training):
  - `RandomRotation(0.06)`: Applica una rotazione di circa $\pm 22°$, simulando la naturale inclinazione della grafia.
  - `RandomZoom(-0.15, 0.05)`: Applica uno zoom asimmetrico. È più probabile che una lettera venga scritta piccola (zoom out fino al 15%) piuttosto che sbordare fuori dalla cella (zoom in massimo del 5%).
  - `RandomTranslation(0.08, 0.08)`: Trasla l'immagine di circa ± 2 pixel per simulare una centratura imperfetta.

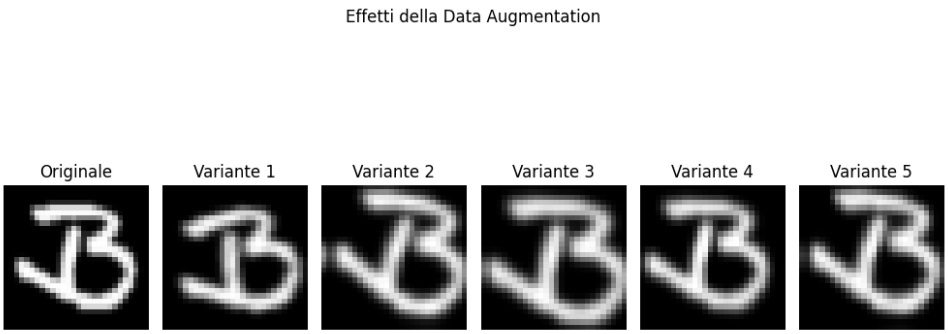

Figura 1: Data Augmentation applicata alla lettera: B

- **Primo Blocco Convoluzionale (Estrazione di basso livello):**
  - **Conv2D (32 filtri 3×3):** I filtri scorrono sull'immagine per individuare feature semplici, come bordi, angoli e linee. L'uso del `padding='same'` mantiene inalterata la dimensione spaziale (28×28). Viene usata la funzione di attivazione **ReLU** (Rectified Linear Unit) che azzera i valori negativi, introducendo non-linearità.
  - **MaxPooling2D (2×2):** Riduce la dimensionalità tenendo solo il valore massimo in ogni finestra 2×2. Questo passaggio comprime la feature map a 14×14, riducendo il carico computazionale ed estraendo i pattern più forti.
- **Secondo Blocco Convoluzionale (Estrazione di medio livello):**
  - **Conv2D (64 filtri 3×3):** Aumentando i filtri, la rete impara a combinare i bordi trovati dal livello precedente in forme più complesse (es. curve chiuse, intersezioni di tratti). L'output resta 14×14.
  - **MaxPooling2D (2×2):** Dimezza nuovamente la risoluzione spaziale, portando la mappa a 7×7.
- **Terzo Blocco Convoluzionale (Rappresentazione ad alto livello):**
  - **Conv2D (64 filtri 3×3):** In questa fase, le forme vengono combinate in rappresentazioni di alto livello (la struttura logica della lettera). Si è scelto di mantenere il numero di filtri inalterato rispetto al blocco precedente (creando un bottleneck o collo di bottiglia) per limitare la capacità espressiva della rete: questo previene attivamente il rischio di overfitting su un dominio ristretto a sole 4 classi e riduce drasticamente il peso computazionale del modello.
  - *Nota progettuale:* Qui **non** è stato inserito un ulteriore layer di Pooling. Essendo la dimensione arrivata a soli 7×7 pixel, un'ulteriore riduzione spaziale avrebbe distrutto preziose informazioni strutturali prima della classificazione.
- **Classificatore Finale (Fully Connected):**
  - **Flatten:** Appiattisce le feature map tridimensionali ($7 \times 7 \times 64$) in un unico array monodimensionale di 3136 elementi. La rete ora smette di guardare la "posizione" spaziale dei tratti e si concentra solo sulla loro "presenza".

- **Dense (128 neuroni, ReLU):** Ogni neurone è collegato a tutti i 3136 input. Funge da "riassunto" delle feature estratte, imparando le correlazioni logiche per distinguere le lettere.
- **Dropout (0.4):** Durante l'addestramento, il 40% dei neuroni di questo strato viene "spento" casualmente. Questo obbliga la rete a non affidarsi a singoli percorsi mnemonici, prevenendo in modo efficace l'overfitting.
- **Dense di Output (4 neuroni, Softmax):** L'ultimo strato possiede un neurone per ogni classe (B, G, T, Y). La funzione di attivazione **Softmax** converte i valori in una distribuzione di probabilità la cui somma è 1. Il neurone con la probabilità maggiore decreta la predizione finale per quella cella.

La rete viene infine compilata utilizzando la funzione di costo `categorical_crossentropy` (standard per problemi di classificazione multiclasse) e l'algoritmo di ottimizzazione **Adam**, noto per la sua convergenza rapida e stabile, specialmente in presenza di pesanti layer di Data Augmentation.

4 Image processing e costruzione degli stati

La seconda fase del progetto prevede l'estrazione delle informazioni strutturali e visive a partire dall'immagine della griglia fornita in input. Per raggiungere questo obiettivo, l'intero modulo di visione artificiale è stato progettato e sviluppato sfruttando le potenzialità della libreria OpenCV. Lo scopo è duplice: isolare le singole celle che compongono la griglia e preparare il contenuto di ciascuna cella per renderlo compatibile con il formato del dataset EMNIST richiesto dalla rete neurale.

4.1 Estrazione della griglia mediante Morfologia Matematica

Il primo ostacolo affrontato è stato il riconoscimento automatico della struttura a griglia, a prescindere dallo spessore delle linee disegnate. Il processo implementato segue questi step:

- **Binarizzazione iniziale:** L'immagine viene convertita in scala di grigi e sottoposta a un *threshold* inverso a soglia fissa (valore 200 su 255). In questo modo, le linee scure della griglia diventano pixel bianchi su uno sfondo completamente nero, facilitando l'analisi spaziale.

- **Rilevamento morfologico:** Per isolare le linee orizzontali e verticali sono state applicate operazioni di morfologia matematica, in particolare l'apertura (`MORPH_OPEN`). Sono stati creati due kernel rettangolari dinamici, proporzionali alle dimensioni dell'immagine (1/10 della larghezza per le orizzontali e 1/10 dell'altezza per le verticali). Questo passaggio isola l'ossatura della griglia filtrando gli elementi interni.

- **Intersezione logica:** Eseguendo un'operazione di `bitwise_and` (AND logico) tra l'immagine delle linee verticali e quella delle orizzontali, si ottengono unicamente i punti di intersezione, ovvero i vertici fisici delle celle.

- **Clustering delle coordinate:** Poiché una linea spessa genera multipli pixel di intersezione vicini tra loro, è stata implementata la funzione `cluster_positions`: essa ordina i pixel trovati e raggruppa quelli distanti meno di una certa soglia (es. 10 pixel) l'uno dall'altro. Il risultato è un elenco pulito ed esatto delle coordinate X e Y che delimitano le righe e le colonne.

4.2 Ritaglio intelligente e normalizzazione (Auto-Crop)

Una volta note le coordinate, l'immagine originale viene divisa in singole celle (Region of Interest o ROI). Tuttavia, passare la ROI grezza alla rete neurale porterebbe a predizioni errate, poiché le lettere potrebbero essere decentrate o di dimensioni non standard. Per risolvere questo problema, la funzione `auto_crop_cell` elabora le celle nel seguente modo:

1. **Metodo di Otsu Inverso:** La cella viene binarizzata usando il metodo di Otsu (`THRESH_OTSU`), che calcola automaticamente la soglia ottimale in base all'istogramma dei grigi della specifica cella, facendo risaltare il tratto della lettera su sfondo nero.

2. **Bounding Box del contorno maggiore:** Tramite `findContours` (`RETR_EXTERNAL`) si individuano tutte le forme nella cella. Viene selezionato solo il contorno con l'area maggiore (per ignorare l'eventuale rumore) e se ne estrae il rettangolo di delimitazione minimo (*Bounding Box*).

3. **Canvas quadrato e Padding:** Viene creato un "canvas" nero quadrato. Il lato di questo canvas è pari al lato maggiore del bounding box maggiorato di un *padding* del 20%. La lettera estratta viene incollata esattamente al centro del canvas. Questo artificio è cruciale per replicare il formato in cui le lettere EMNIST sono centrate e spaziate dai bordi.

4. **Ridimensionamento EMNIST:** L'immagine perfettamente centrata e quadrata viene infine ridimensionata a 28×28 pixel usando un'interpolazione ad area (`INTER_AREA`), ottima per la riduzione delle dimensioni.

Le celle così elaborate vengono raccolte in un unico *batch* di array Numpy, normalizzate dividendole per 255.0 e passate alla CNN in una singola chiamata predittiva, ottimizzando drasticamente i tempi di esecuzione. A valle della predizione, un blocco logico verifica la corretta rilevazione della testina "T"; in assenza di quest'ultima, l'esecuzione viene interrotta notificando l'anomalia, garantendo la robustezza dell'input prima di avviare l'algoritmo di ricerca.

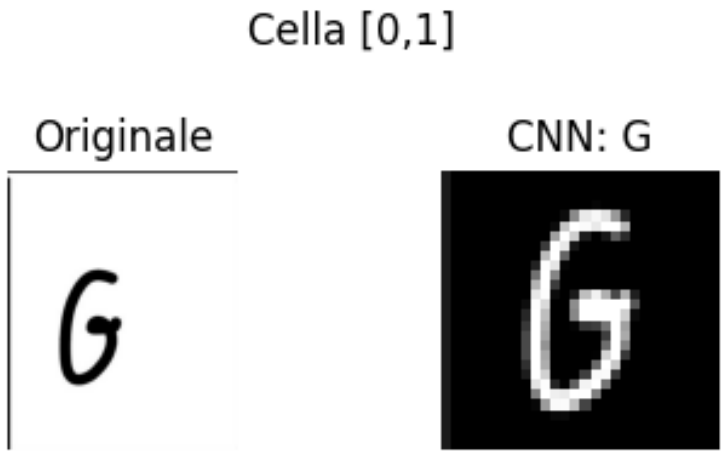

Figura 2: Image Processing

5 Algoritmi di ricerca e Solutore del Problema

5.1 Modellazione del dominio (Problem)

Il problema è stato modellato implementando la classe `UniformColoring`, derivata dalla classe base `Problem` della libreria AIMA. Lo stato è rappresentato dalla classe `ColoringState`, caratterizzata da:

- `grid`: Una tupla che rappresenta i colori di tutte le celle (immutabile e quindi *hashable*).
- `held_color`: Il colore della cella attualmente nascosto sotto la testina 'T'.
- `t_pos`: L'indice precalcolato della posizione corrente della testina.

I metodi principali sono stati così ridefiniti:

- `actions`: Genera le azioni valide (N, S, W, E) verificando i bordi della griglia, e le azioni di colore (COL-B, COL-Y, COL-G) solo se non ci si trova sulla *start position*.
- `result`: Dato uno stato e un'azione, ritorna lo stato successivo aggiornando la posizione, il colore della cella liberata e il nuovo `held_color`.
- `path_cost`: Associa alle azioni i costi differenziati previsti dal dominio.
- `goal_test`: Verifica che la testina si trovi nella posizione di partenza e che tutte le celle (escludendo 'T' e 'None') abbiano esattamente lo stesso colore.

5.2 Euristica (h_combined_cost)

Per ottimizzare la ricerca è stata definita un'euristica composita `h_combined_cost` così calcolata:

$$h(n) = h_{dist}(n) + h_{paint}(n)$$

- h_{dist}: Rappresenta la distanza spaziale di Manhattan dalla posizione attuale della testina alla sua casella di partenza (Start Position). Poiché il dominio impone che la testina debba tornare alla base per soddisfare il goal test, questo percorso è inevitabile e il suo costo (1 per passo) è certo. Per ragioni di efficienza computazionale, le coordinate di griglia (X e Y) non vengono ricalcolate tramite divisioni ad ogni espansione, ma vengono estratte da tuple di lookup (tabelle di ricerca) precalcolate in fase di inizializzazione, riducendo drasticamente i tempi di esecuzione.
- h_{paint}: È una stima del costo teorico minimo per uniformare tutte le celle visibili. L'algoritmo estrae tutti i colori attualmente presenti nella griglia (ignorando le celle vuote 'None' e la posizione 'T') e vi aggiunge logicamente il colore attualmente nascosto sotto la testina (`held_color`). Successivamente, simula il costo per trasformare tutte queste celle nel colore Blu ($cost_B$), Giallo ($cost_Y$) o Verde ($cost_G$). Il valore di h_{paint} corrisponderà al minimo assoluto tra questi tre scenari: $min(cost_B, cost_Y, cost_G)$.

Ammissibilità e consistenza: L'euristica è **ammissibile** in quanto non sovrastima mai il costo reale per raggiungere il goal. Essa somma la distanza fisica minima certa (h_{dist}) ai costi puri del pigmento (h_{paint}), ignorando del tutto i movimenti supplementari necessari per spostarsi fisicamente da una cella da colorare all'altra. Questa voluta "cecità" ai costi di spostamento intermedi garantisce che la stima si mantenga sempre inferiore o uguale al costo effettivo, assicurando che l'algoritmo A* mantenga la sua garanzia di ottimalità.

Analisi delle varianti:

- Se il costo di spostamento spaziale fosse pari a 0, l'euristica perderebbe l'ammissibilità perché h_{dist} aggiungerebbe un costo positivo a fronte di un costo reale nullo per i movimenti.
- Se i costi di colorazione fossero uniformi, h_{paint} rimarrebbe valida: basterebbe moltiplicare il numero di celle "diverse" dal colore target per tale costo base, mantenendo la sua natura di stima per difetto.

5.3 Algoritmi utilizzati

Sono stati eseguiti e confrontati tre algoritmi di ricerca su grafo:

- **Uniform Cost Search (UCS)**: Ricerca non informata, garantisce ottimalità.
- **Greedy Best First Search**: Ricerca informata, si guida solo sull'euristica, rapida ma non garantisce il costo minimo.
- **A* Search**: Ricerca informata ($f(n) = g(n) + h(n)$), sfrutta l'euristica garantendo la soluzione ottima in presenza di euristiche ammissibili.

6 Esecuzione e Statistiche dei Risultati

6.1 Esecuzione del programma

L'esecuzione del programma principale è progettata per fornire un riscontro a video chiaro e dettagliato di ogni fase del processo, permettendo all'utente di monitorare sia il corretto funzionamento della visione artificiale, sia le performance dei singoli algoritmi. L'output a terminale si struttura nei seguenti passaggi sequenziali:

1. **Inizializzazione e Rilevamento:** All'avvio, il sistema notifica l'avvenuto caricamento del modello di rete neurale. Subito dopo, stampa a video la dicitura `=== GRIGLIA INIZIALE RILEVATA ===` seguita dalle dimensioni rilevate (es. 2x4) e dalla rappresentazione testuale della matrice. Questo passaggio è cruciale perché permette all'utente di verificare visivamente che la CNN abbia classificato correttamente tutte le lettere ('B', 'G', 'Y') e la testina ('T') prima dell'inizio della ricerca.
2. **Esecuzione sequenziale degli algoritmi:** Il programma esegue in successione Uniform Cost Search (UCS), Greedy Best First Search e A* Search. Per ogni algoritmo, l'output è chiaramente separato e formattato, mostrando un report completo delle prestazioni:
   - **Statistiche di ricerca:** Vengono stampati i valori di `Total nodes generated` (nodi totali creati in memoria), `Paths explored` (nodi estratti dalla frontiera ed espansi) e `Nodes left in frontier` (nodi in attesa, utile per valutare l'occupazione di memoria).
   - **Dettagli della soluzione:** Il campo `Result` mostra la lista sequenziale delle azioni ottime (movimenti cardinali e azioni di colorazione) necessarie per raggiungere il goal. Vengono inoltre indicati esplicitamente il `Path Cost` (il costo totale cumulato) e la `Path Length` (il numero di passi effettuati).
   - **Verifica visiva dello stato finale:** Sotto la voce `=== MATRICE FINALE RISOLTA ===`, il programma stampa la configurazione finale della griglia. Questo permette di constatare immediatamente la correttezza della soluzione: tutte le celle mostreranno lo stesso colore (il colore target scelto dall'algoritmo) e la testina 'T' si troverà esattamente nella posizione da cui era partita.
   - **Tempo di esecuzione:** Ogni blocco si chiude con il calcolo del tempo netto impiegato per trovare la soluzione, espresso in secondi.
3. **Simulazione visiva dell'esecuzione (GIF):** A completamento della ricerca è stato implementato un modulo di simulazione grafica automatizzata. Utilizzando le librerie `matplotlib` e `imageio`, il programma elabora il percorso ricavato da ciascun algoritmo e genera un'animazione in formato GIF (salvata automaticamente nella cartella `gif/`, ad esempio `simulazione_astar.gif`). Questa sequenza di immagini mostra frame per frame l'evoluzione della griglia: viene rappresentata la transizione degli stati dopo ogni azione e

la testina 'T' viene evidenziata in rosso per far risaltare il suo spostamento spaziale. Tale funzionalità permette una validazione visiva, immediata e intuitiva dei piani di esplorazione calcolati.

Nota di Debug Visivo: All'interno dello script è presente un parametro `verbose` (impostabile a `True` nel richiamo della funzione di estrazione). Attivando questa opzione, durante la fase di image processing verranno mostrate a schermo delle interfacce grafiche che affiancano i ritagli originali delle singole celle all'output predittivo della CNN. Questa funzionalità risulta estremamente preziosa per diagnosticare eventuali errori di lettura e per ispezionare visivamente e in tempo reale ciò che la rete neurale sta effettivamente elaborando.

```
[INFO] Modello 'cnn_coloring_project.keras' caricato con successo.
[INFO] Griglia rilevata: 2x4 (8 celle)

=== GRIGLIA INIZIALE RILEVATA (2x4) ===
('T', 'G', 'G', 'B')
('Y', 'G', 'Y', 'B')

============================================================
Uniform Cost Search (UCS)

Total nodes generated:   9046
Paths explored:          2044
Nodes left in frontier:  1674
Result: ['E', 'COL-B', 'E', 'COL-B', 'S', 'COL-B', 'W', 'COL-B', 'W', 'COL-B', 'N']
Path Cost:   11
Path Length: 11

=== MATRICE FINALE RISOLTA ===
('T', 'B', 'B', 'B')
('B', 'B', 'B', 'B')

Time: 2.1153 s
============================================================
Greedy Best First Search

Total nodes generated:   151
Paths explored:          39
Nodes left in frontier:  48
Result: ['S', 'COL-B', 'N', 'E', 'COL-B', 'S', 'COL-B', 'N', 'E', 'COL-B', 'S', 'COL-B', 'W', 'W', 'N']
Path Cost:   15
Path Length: 15

=== MATRICE FINALE RISOLTA ===
('T', 'B', 'B', 'B')
('B', 'B', 'B', 'B')

Time: 0.0000 s
============================================================
A* Search

Total nodes generated:   795
Paths explored:          175
Nodes left in frontier:  194
Result: ['E', 'COL-B', 'E', 'COL-B', 'S', 'COL-B', 'W', 'COL-B', 'W', 'COL-B', 'N']
Path Cost:   11
Path Length: 11

=== MATRICE FINALE RISOLTA ===
('T', 'B', 'B', 'B')
('B', 'B', 'B', 'B')

Time: 0.0218 s
```

Figura 3: Esempio esecuzione.

6.2 Risultati Rete Neurale

Durante il training (8 epoche), il modello ha mostrato i seguenti risultati:

- **Test Accuracy: 98.66%**
- **Test Loss: 0.0515**

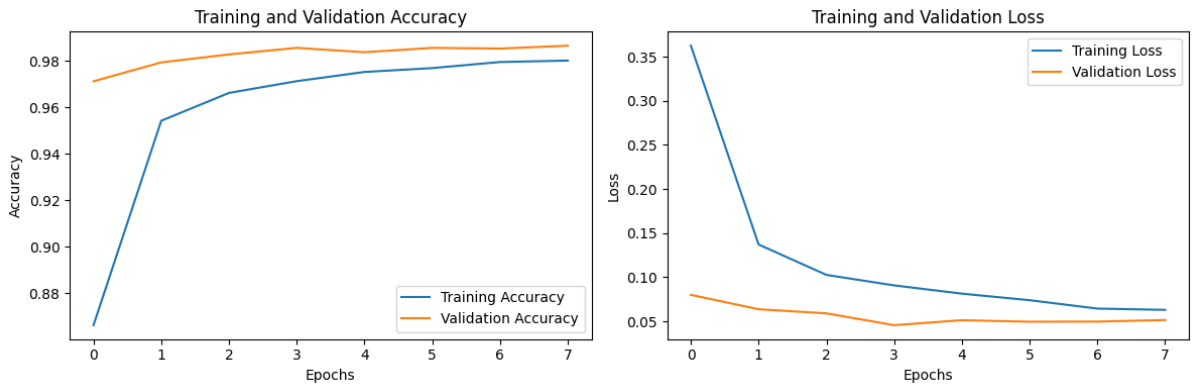

Figura 4: Grafici di Model Accuracy e Model Loss.

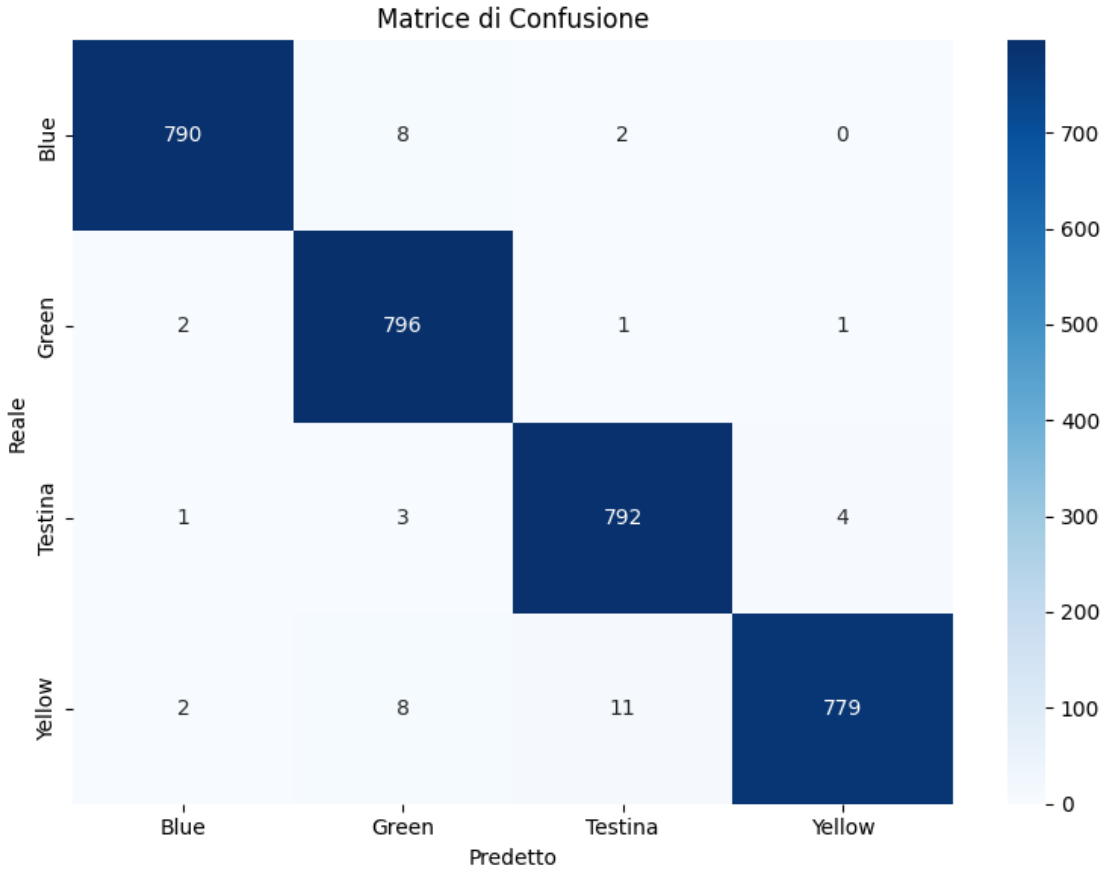

Figura 5: Confusion Matrix.

Il modello si è rivelato estremamente robusto nel classificare le celle reali generate dall'algoritmo di visione artificiale, grazie anche alla data augmentation applicata in fase di addestramento.

6.3 Prestazioni degli Algoritmi di Ricerca

Di seguito verrà analizzato il comportamento dei vari algoritmi al variare della grandezza dello spazio degli stati (al variare della dimensione della matrice).

6.3.1 Matrice 2x2

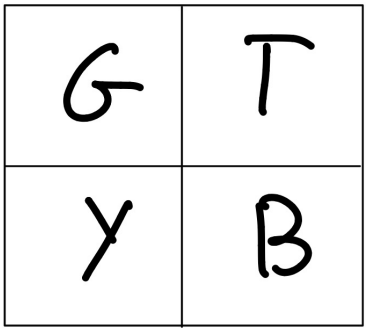

Figura 6: Stato iniziale matrice 2x2.

Algoritmo	Nodi generati	Percorsi esplorati	Costo Path	Lunghezza Path	Tempo (s)
UCS	*113*	*31*	*6*	*6*	*0.0010*
Greedy	*39*	*11*	*6*	*6*	*0.0010*
A*	*33*	*9*	*6*	*6*	*0.0001*

Data la ridotta complessità dello spazio degli stati per la griglia 2x2, tutti gli algoritmi convergono facilmente verso la medesima soluzione ottima. La differenza sostanziale risiede nell'efficienza dell'esplorazione: UCS, basandosi su una ricerca non informata, è costretto a espandere un numero di nodi nettamente superiore, mentre A e Greedy sfruttano l'euristica per raggiungere il goal in modo molto più diretto.*

6.3.2 Matrice 2x3

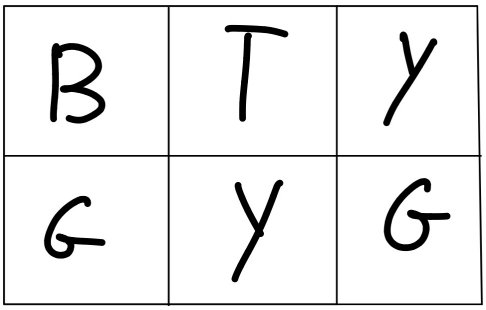

Figura 7: Stato iniziale matrice 2x3.

Algoritmo	Nodi generati	Percorsi esplorati	Costo Path	Lunghezza Path	Tempo (s)
UCS	*1803*	*443*	*10*	*10*	*0.0673*
Greedy	*89*	*22*	*12*	*12*	*0.0001*
A*	*395*	*96*	*10*	*10*	*0.0166*

Con uno spazio degli stati leggermente più complesso, possiamo notare come UCS richieda molti nodi per garantire la soluzione ottima. L'algoritmo Greedy è molto veloce ed espande pochissimi nodi, portando però a una soluzione sub-ottimale. A si conferma la strategia vincente, trovando il medesimo ottimo di UCS ma esplorando molti meno percorsi.*

6.3.3 Matrice 2x4

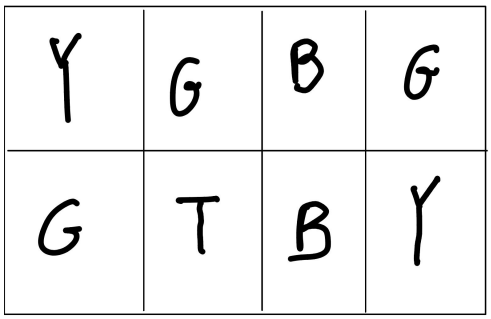

Figura 8: Stato iniziale matrice 2x4.

Algoritmo	Nodi generati	Percorsi esplorati	Costo Path	Lunghezza Path	Tempo (s)
UCS	*12770*	*2971*	*13*	*13*	*3.3448*
Greedy	*406*	*98*	*17*	*17*	*0.0156*
A*	*756*	*179*	*13*	*13*	*0.0202*

Con l'ulteriore espansione dello spazio degli stati nella griglia 2x4, l'esplosione combinatoria diventa evidente: UCS necessita di esplorare quasi 3000 percorsi (impiegando oltre 3 secondi) per garantire l'ottimo globale (Costo 13). In questo scenario più dispersivo, l'algoritmo Greedy risulta estremamente rapido ma commette gravi errori strategici, restituendo una soluzione ampiamente sub-ottimale (Costo 17). A domina nuovamente il confronto: individua la medesima soluzione ottima di UCS ma abbattendo drasticamente i percorsi esplorati da 2971 a soli 179, confermando la notevole efficacia e scalabilità dell'euristica progettata.*

6.3.4 Matrice 2x5

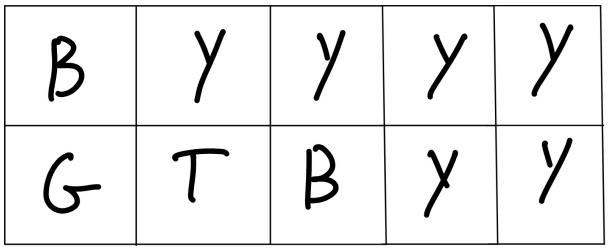

Figura 9: Stato iniziale matrice 2x5.

Algoritmo	Nodi generati	Percorsi esplorati	Costo Path	Lunghezza Path	Tempo (s)
UCS	*24839*	*5540*	*12*	*9*	*18.0513*
Greedy	*1613*	*374*	*23*	*23*	*0.1146*
A*	*386*	*89*	*12*	*9*	*0.0070*

Raggiungendo la dimensione 2x5, l'esplosione combinatoria porta la ricerca non informata al proprio limite: UCS impiega ben 18 secondi e genera quasi 25.000 nodi per individuare il percorso ottimo (Costo 12). In uno spazio di ricerca così ampio, le scelte miopi dell'algoritmo Greedy degenerano in una soluzione gravemente sub-ottimale, con un costo quasi doppio rispetto all'ottimo (Costo 23 contro 12). A esprime in questo test la sua massima efficienza: grazie*

a un'eccellente potatura dei rami morti, esplora appena 89 percorsi (generando solo 386 nodi) e trova la soluzione ottima in soli 0.007 secondi. È emblematico notare come, in questo scenario, l'accuratezza dell'euristica renda A nettamente più veloce dello stesso Greedy, evitando che l'algoritmo si disperda nell'esplorazione di un percorso inutilmente lungo (lunghezza 9 per A* contro 23 per Greedy).*

6.3.5 Matrice 3x4

B	G	T	B
G	Y	B	G
Y	G	Y	Y

Figura 10: Stato iniziale matrice 3x4.

Algoritmo	Nodi generati	Percorsi esplorati	Costo Path	Lunghezza Path	Tempo (s)
UCS	-	-	-	-	-
Greedy	*2643*	*573*	*24*	*24*	*0.2769*
A*	*11124*	*2314*	*18*	*18*	*4.7631*

Con la matrice 3x4 l'esplosione combinatoria rende impraticabile la ricerca cieca: dopo oltre 10 minuti di elaborazione ininterrotta, UCS è stato interrotto senza riuscire a trovare una soluzione. Questo fallimento dimostra l'assoluta necessità dell'euristica progettata. A si conferma infatti l'unico algoritmo capace di garantire la soluzione ottima (Costo 18) in tempi ragionevoli (4.7 secondi), mentre Greedy, seppur fulmineo, si arrende a un percorso gravemente sub-ottimale (Costo 24).*